

CSCE 465 — Homework 2

Protect Agent Messages with Classic Cryptography

Date: September 30, 2026

For this assignment, I tested what encryption protects and what extra checks are needed to detect changes and replays. I ran the experiments in my Ubuntu course VM.

Lab Preparation

I used the hw2 folder inside my existing csce465-agentsec directory. The Python virtual environment was in the parent folder. My environment had the following versions:

```
Python 3.12.3
OpenSSL 3.0.13 30 Jan 2024
cryptography 49.0.0
pytest 9.1.1
DH Parameters: (3072 bit)
GROUP: ffdhe3072
```

I generated ffdhe3072.pem with OpenSSL and checked that it showed a 3072-bit group named ffdhe3072. Before the setup worked, I had to install the Python venv package and fix the VM clock. I saved the environment output in lab_preparation.txt and included a screenshot in the appendix.

Task 1 — Encryption Alone Is Insufficient

Design

baseline_ctr.py encrypts the command {"action":"READ","path":"notes.txt"} using AES-256-CTR. It generates a random 32-byte key and a random 16-byte IV. The receiver only records the decrypted command, so the experiment does not actually change any files. The relay knows which bytes say READ and what to replace them with, but it does not have the key. This first version has no MAC or replay check.

Observed Results

In the first test, the relay changed READ to EDIT. Both strings are four bytes long, so the replacement did not change the message length. The XOR calculation was:

```

READ:          52 45 41 44
EDIT:          45 44 49 54
XOR difference: 17 01 08 10

```

After the relay applied this difference to the ciphertext, the receiver read EDIT instead of READ. I then sent the same modified ciphertext and IV again. The receiver accepted the command a second time:

```

Processed #1: {"action":"EDIT","path":"notes.txt"}
Processed #2: {"action":"EDIT","path":"notes.txt"}
Replay accepted: the same command was processed twice.

```

Explanation

This works because CTR encrypts plaintext by XORing it with a keystream: $C = P \text{ XOR } S$. If the attacker changes the ciphertext to $C' = C \text{ XOR } P \text{ XOR } P'$, decrypting it produces P' . The attacker only needs to know the original bytes and where they appear, not the encryption key. The receiver accepted the changed command because there was no MAC to detect the modification. It accepted the replay because it did not track which messages it had already processed. These are two separate problems: adding a MAC would detect tampering, but a valid message could still be replayed without a sequence check.

Evidence: baseline_ctr.py, task1_output.txt, and Figure 2 in the appendix.

Task 2 — Authenticated Diffie–Hellman Setup

Design and Canonical Encoding

In handshake.py, the gateway and node communicate inside one Python program, so no network sockets are needed. Each side has a long-term 3072-bit RSA signing key and a trusted copy of the other side's public key. Each new session gets a fresh DH private key and a random 16-byte nonce. The DH keys use the ffdhe3072 file from the setup step. RSA is used to sign the handshake, not to encrypt the later messages.

The transcript puts the fields in one fixed order: CSCE465-HS-v2, ffdhe3072, gateway identity, node identity, gateway DH public value, node DH public value, gateway nonce, and node nonce. Each field starts with its length as a four-byte big-endian integer. This makes the field boundaries clear. Both

public values are stored as 384-byte big-endian values. Before hashing, the parser rejects incorrect lengths, missing or extra data, wrong labels, and unexpected identities.

Each side first checks that its own public value and nonce are still the ones it generated. It then signs role || SHA-256(transcript) with RSA-PSS and SHA-256. To check the other side, it uses the trusted peer public key and the expected peer role. The accepted transcript must match the one it signed. Once finish succeeds or fails, that handshake attempt cannot be reused. The cryptography library handles the signatures and DH operations.

Exact Key Derivation

I used the key derivation given in the assignment. TH is the SHA-256 hash of the transcript, and Z is the DH shared secret written as 384 big-endian bytes, with zeros added on the left if needed:

```
K_master = SHA-256("CSCE465-KDF-v1" || Z || TH)
K_g2n_enc = HMAC-SHA-256(K_master, "gateway-to-node encryption" || TH)
K_g2n_mac = HMAC-SHA-256(K_master, "gateway-to-node MAC" || TH)
K_n2g_enc = HMAC-SHA-256(K_master, "node-to-gateway encryption" || TH)
K_n2g_mac = HMAC-SHA-256(K_master, "node-to-gateway MAC" || TH)
session_id = HMAC-SHA-256(K_master, "session identifier" || TH)[:8]
```

Observed Results and Security Concepts

The handshake run produced this output:

```
Handshake accepted by both parties.
Both parties derived identical session keys.
Four distinct 32-byte traffic keys derived.
Session ID: f8e0fd3dd1840fe2
Second session derived fresh keys and a different session ID.
```

DH lets both sides calculate the same secret. The signatures give each side evidence that the exchange came from the expected identity. Including both identities prevents accepting the wrong peer, while including the role stops a gateway signature from being used as a node signature. The fresh nonces and public values connect the transcript to the current session, so an old transcript is rejected. Fresh DH private keys also provide forward secrecy: obtaining an RSA signing key later should not reveal old session keys if the old DH secrets are gone. Removing a Python reference does not guarantee that the secret has been erased from memory.

The different derivation labels produce separate keys for encryption, MACs, and each direction. Without the signatures, an attacker could replace the DH public values and create one shared secret with the gateway and another with the node. The attacker could then read and change traffic while forwarding it between them. Fresh nonces would not stop that attack because they do not prove identity.

The first handshake test run passed all 17 cases in 74.37 seconds. These tests checked normal key agreement and several ways an attacker could alter or reuse the handshake. Passing them shows that those cases behaved as expected, but it does not prove that every possible attack is covered. The code also assumes the group file is the trusted one generated during setup; checking its bit length and generator alone does not identify the exact standardized prime.

Evidence: handshake.py, ffdhe3072.pem, tests/test_handshake.py, task2_output.txt, task2_tests.txt, and Figures 3–4.

Task 3 — Encrypt-then-MAC Record Layer

Format and State

secure_record.py has two main functions: seal() creates a protected record, and open_record() checks and decrypts it. Each record contains header || IV || ciphertext || tag. The header is 15 bytes: one byte for the version, one for the direction, eight for the sequence number, one for the message type, and four for the ciphertext length. The numbers use big-endian encoding. Version is 1, direction 0 is gateway-to-node, and direction 1 is node-to-gateway.

```
IV = session_id(8 bytes) || sequence(8 bytes)
ciphertext = AES-256-CTR(K_enc, IV, plaintext)
tag = HMAC-SHA-256(K_mac, header || IV || ciphertext)
```

Both directions start their sequence numbers at zero and use different encryption and MAC keys. The sender reserves a sequence number before encrypting and refuses to wrap around after the maximum value. Locks keep two operations from using the same counter at the same time. The state must stay with its session; resetting the counter while keeping the same keys could reuse an IV. If sending fails after reserving a number, a new session may be needed because the receiver expects the exact next number.

Verification and Observed Results

The receiver first checks the record length and verifies the MAC using the library's constant-time verification function. It then checks the version, direction, sequence number, and IV. Decryption only happens after every check passes. If anything fails, no plaintext is returned and the receive counter stays unchanged. A reflected record fails MAC verification because the other direction uses a different key.

The record-layer demonstration produced:

```

Node received: {"action":"READ","path":"notes.txt"}
Gateway received: {"status":"OK"}
Replay rejected: unexpected sequence number
Modified ciphertext rejected: invalid record MAC
Original next record accepted after tampering was rejected.
Reflected record rejected: invalid record MAC
Task 3 demonstration completed.

```

There is a limitation in the assignment's IV formula. For a record longer than one AES block, CTR can reach the same counter value used at the start of the next record. Even though the starting IVs differ, some keystream bytes can repeat. This implementation keeps the required formula. I discuss the resulting confidentiality problem in the security note.

Evidence: `secure_record.py`, `task3_output.txt`, and Figure 5.

Task 4 — Adversarial Tests

I ran the full test suite in Ubuntu with `python3 -m pytest -v tests`. The result was 35 passed in 78.17 seconds. This includes 17 handshake cases and 18 record-layer cases, counting each parametrized case separately.

The required attack tests gave the following results:

- Valid handshake and bidirectional messages: matching derived keys and correct plaintext/type in both directions.
- Modified ciphertext: rejected with invalid record MAC; receive state remains unchanged.
- Modified authenticated header: message-type alteration rejected with invalid record MAC. Separate tests recompute a tag using test-only key access to exercise authenticated invalid version and direction checks.
- Replayed record: rejected with unexpected sequence number after first acceptance.
- Reflected record: rejected with invalid record MAC at the originating endpoint.
- Incorrect RSA key, invalid signature, and reflected handshake: rejected with invalid peer signature; additional tests isolate signed-role binding and reflected DH offers.

The other record tests cover changed IVs and tags, incomplete records, extra bytes, out-of-order messages, empty messages, IV formatting, counter exhaustion, and separate directional keys. To check verify-before-decrypt, the tampering tests replace the Cipher constructor with a function that fails if it is called. The tests still pass because the bad MAC is rejected before the code tries to decrypt.

Some record tests reuse fixture keys and reset counters between isolated test cases to save time. A real

session must not do this. These results show that the tested rejection rules work, but they do not fix the CTR counter-overlap problem or prove that the whole protocol is secure.

Evidence: tests/test_handshake.py, tests/test_secure_record.py, all_tests_output.txt, and Figure 6.

Security Note

The attacker in this lab can observe and modify messages between the gateway and node, including changing, injecting, delaying, reordering, reflecting, and replaying them. I assume the attacker has access neither to either endpoint's private rsa key nor to the session keys. Each side has a trusted copy of the other's public key already installed. The protocol offers no protection for compromised endpoints, hides nothing about traffic patterns, guarantees nothing about delivery or the appropriateness of any requested action.

The handshake uses fresh dh private values and random nonces for each session. Both sides sign the same transcript, containing their identities, public values, and nonces; the signed role indicates to the verifier which signature corresponds to the gateway and which to the node, impeding an attacker's ability to substitute a different exchange or reflect a signature back to its sender. Checking the local nonce and public value also prevents presenting an old transcript in a new session. Forward secrecy relies on old dh secrets being unavailable; dropping a python reference is not sufficient for erasing a value from memory.

CTR encryption relies on uniqueness of the keystream; XORing two ciphertexts encrypted with the same keystream reveals the XOR of their plaintexts, potentially allowing known-plaintext attacks. Thus, every AES counter block must be unique per key. There is an issue with the assignment's particular IV format: the session_id || sequence provides consecutive starting counters for consecutive records. The second block of record zero uses the same counter as the first block of record one. Records spanning more than 16 bytes may have overlapping keystreams with following records. This implementation uses the required formula, but varying the starting IV does not address the possibility of nonoverlapping counter ranges. A repaired design might use separate record and block-counter fields, with maximum sizes enforced to prevent overflow.

The receiver parses the HMAC and verifies it before decrypting; this prevents unauthenticated ciphertext from reaching the decryption function. The MAC covers the header, IV, and ciphertext, preventing an attacker from modifying the message type, sequence number, or encrypted contents without detection. The library performs the tag check using a constant-time comparison; the length check occurs first, but a failed record never produces plaintext. The receiver also requires that a received record have the exact next sequence number, preventing replays and out-of-order records. A record with an invalid sequence number does not advance the counter.

Encryption and authentication use different keys, since they serve different purposes; each direction of traffic also has its own pair. A gateway record reflected back to the gateway would fail verification under

the node-to-gateway mac key. The assignment's use of different derivation labels provides this separation. Counters cannot be reset while using the same session keys; restarting the communication requires a new handshake.

AES-GCM is one example of an authenticated encryption with associated data, or AEAD, scheme. These provide encryption and authentication in a single step and can include header information as associated data, reducing the number of cryptographic operations an application must perform. However, it is still necessary to provide the proper nonce to GCM and for the receiver to enforce a replay policy; using a AEAD does not eliminate the need for sequence tracking.

Finally, an authenticated message could still request a dangerous action. A compromised gateway could generate a record requesting a destructive tool call with a valid MAC, for instance; the MAC would authenticate that the record was generated by an entity with the session key, but that does not indicate that the requested action was authorized. The application still needs to implement access controls, input sanitation, limited tool capabilities, and isolation. The receiver in this lab only tracks harmless simulated commands; authenticating the message does not replace authorization logic for what the endpoint is allowed to do.

References

- Course assignment: HW2-2.pdf, provided locally.
- cryptography 49.0.0, Diffie–Hellman: <https://cryptography.io/en/49.0.0/hazmat/primitives/asymmetric/dh/>
- cryptography 49.0.0, RSA: <https://cryptography.io/en/49.0.0/hazmat/primitives/asymmetric/rsa/>
- cryptography 49.0.0, symmetric encryption: <https://cryptography.io/en/49.0.0/hazmat/primitives/symmetric-encryption/>
- cryptography 49.0.0, HMAC: <https://cryptography.io/en/49.0.0/hazmat/primitives/mac/hmac/>

Evidence Appendix

The following screenshots document the experiments and test results from the Ubuntu course VM.

1. Lab preparation: evidence/01_lab_preparation.png
2. Task 1 bit flipping and replay: evidence/02_task1.png
3. Task 2 accepted handshake: evidence/03_task2.png
4. Initial 17 handshake tests: evidence/04_handshake_tests.png
5. Task 3 record demonstration: evidence/05_task3.png
6. Final 35-test suite: evidence/06_all_tests.png

Figure 1 — Ubuntu lab preparation

```
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec/hw2$ cat lab_preparation.txt
Python 3.12.3
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
Name: cryptography
Version: 49.0.0
Summary: cryptography is a package which provides cryptographic recipes and primitives to Python developers.
Home-page: https://github.com/pyca/cryptography
Author:
Author-email: The Python Cryptographic Authority and individual contributors <cryptography-dev@python.org>
License-Expression: Apache-2.0 OR BSD-3-Clause
Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: cffi
Required-by:
---
Name: pytest
Version: 9.1.1
Summary: pytest: simple powerful testing with Python
Home-page: https://docs.pytest.org/en/latest/
Author: Brianna Laughner, Bruno Oliveira, Floris Bruynooghe, Freya Bruhin, Holger Krekel, Others (See AUTHORS), Ronny Pfannschmidt
Author-email:
License-Expression: MIT
Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: iniconfig, packaging, pluggy, pygments
Required-by:
    DH Parameters: (3072 bit)
    GROUP: ffdhe3072
```

Figure 2 — CTR alteration and replay

```

baseline_ctr.py --prepare-ctf --prepare-ctf --prepare-ctf --prepare-ctf
ubuntu@ubuntu2404:~/csce465-agentsec/hw2$ source ../.venv/bin/activate
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec/hw2$ wc -l baseline_ctr.py
73 baseline_ctr.py
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec/hw2$ python3 baseline_ctr.py |tee task1_output.txt
Original command: {"action":"READ","path":"notes.txt"}

--- Bit-flip attack: READ -> EDIT ---
Original bytes:      52454144
Replacement bytes: 45444954
XOR difference:     17010810
Original ciphertext: 4ccfb9d1e6e8f73b52c26762a78a20b0e5c493e6ab1666bdaa47168bc96
ae5935a074369
Modified ciphertext: 4ccfb9d1e6e8f73b52c26775a68230b0e5c493e6ab1666bdaa47168bc96
ae5935a074369
Processed #1: {"action":"EDIT","path":"notes.txt"}

--- Replay the identical modified ciphertext ---
Processed #2: {"action":"EDIT","path":"notes.txt"}
Replay accepted: the same command was processed twice.

```

Figure 3 — Authenticated DH handshake

```
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec/hw2$ python3 handshake.py | tee task2_output.txt
Handshake accepted by both parties.
Both parties derived identical session keys.
Four distinct 32-byte traffic keys derived.
Session ID: f8e0fd3dd1840fe2
Second session derived fresh keys and a different session ID.
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec/hw2$
```

Figure 4 — Initial 17 handshake tests

```

rootdir: /home/ubuntu/csce465-agentsec/hw2
collecting ... collected 17 items

tests/test_handshake.py::test_valid_handshake PASSED [ 5%]
tests/test_handshake.py::test_fresh_session_keys PASSED [ 11%]
tests/test_handshake.py::test_changed_local_fields[4-local DH public value changed] PASSED [ 17%]
tests/test_handshake.py::test_changed_local_fields[6-local nonce changed] PASSED [ 23%]
tests/test_handshake.py::test_changed_peer_fields_break_signature[5] PASSED [ 29%]
tests/test_handshake.py::test_changed_peer_fields_break_signature[7] PASSED [ 35%]
tests/test_handshake.py::test_incorrect_declared_length PASSED [ 41%]
tests/test_handshake.py::test_truncated_transcript PASSED [ 47%]
tests/test_handshake.py::test_trailing_transcript_data PASSED [ 52%]
tests/test_handshake.py::test_unexpected_identity PASSED [ 58%]
tests/test_handshake.py::test_invalid_signature PASSED [ 64%]
tests/test_handshake.py::test_incorrect_peer_public_key PASSED [ 70%]
tests/test_handshake.py::test_reflected_signature PASSED [ 76%]
tests/test_handshake.py::test_role_binding_even_with_same_rsa_key PASSED [ 82%]
tests/test_handshake.py::test_reflected_dh_offer PASSED [ 88%]
tests/test_handshake.py::test_old_transcript_rejected_by_new_session PASSED [ 94%]
tests/test_handshake.py::test_completed_handshake_cannot_be_reused PASSED [100%]

===== 17 passed in 74.37s (0:01:14) =====

```

Figure 5 — Record-layer demonstration

```
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec/hw2$ nano secure_record.py
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec/hw2$ python3 secure_record.py 2>&1
| tee task3_output.txt
Node received: {"action":"READ","path":"notes.txt"}
Gateway received: {"status":"OK"}
Replay rejected: unexpected sequence number
Modified ciphertext rejected: invalid record MAC
Original next record accepted after tampering was rejected.
Reflected record rejected: invalid record MAC
Task 3 demonstration completed.
```

Figure 6 — Full suite: 35 tests passed

```

cachedir: .pytest_cache
rootdir: /home/ubuntu/csce465-agentsec/hw2
collecting ... collected 35 items

tests/test_handshake.py::test_valid_handshake PASSED [ 2%]
tests/test_handshake.py::test_fresh_session_keys PASSED [ 5%]
tests/test_handshake.py::test_changed_local_fields[4-local DH public value changed] PASSED [ 8%]
tests/test_handshake.py::test_changed_local_fields[6-local nonce changed] PASSED [ 11%]
tests/test_handshake.py::test_changed_peer_fields_break_signature[5] PASSED [ 14%]
tests/test_handshake.py::test_changed_peer_fields_break_signature[7] PASSED [ 17%]
tests/test_handshake.py::test_incorrect_declared_length PASSED [ 20%]
tests/test_handshake.py::test_truncated_transcript PASSED [ 22%]
tests/test_handshake.py::test_trailing_transcript_data PASSED [ 25%]
tests/test_handshake.py::test_unexpected_identity PASSED [ 28%]
tests/test_handshake.py::test_invalid_signature PASSED [ 31%]
tests/test_handshake.py::test_incorrect_peer_public_key PASSED [ 34%]
tests/test_handshake.py::test_reflected_signature PASSED [ 37%]
tests/test_handshake.py::test_role_binding_even_with_same_rsa_key PASSED [ 40%]
tests/test_handshake.py::test_reflected_dh_offer PASSED [ 42%]
tests/test_handshake.py::test_old_transcript_rejected_by_new_session PASSED [ 45%]
tests/test_handshake.py::test_completed_handshake_cannot_be_reused PASSED [ 48%]
tests/test_secure_record.py::test_bidirectional_messages PASSED [ 51%]
tests/test_secure_record.py::test_tampering_rejected_before_decryption[ciphertext] PASSED [ 54%]
tests/test_secure_record.py::test_tampering_rejected_before_decryption[header] PASSED [ 57%]
tests/test_secure_record.py::test_tampering_rejected_before_decryption[iv] PASSED [ 60%]
tests/test_secure_record.py::test_tampering_rejected_before_decryption[tag] PASSED [ 62%]
tests/test_secure_record.py::test_replay_rejected PASSED [ 65%]
tests/test_secure_record.py::test_reflection_rejected PASSED [ 68%]
tests/test_secure_record.py::test_out_of_order_rejected_without_advancing_state PASSED [ 71%]
tests/test_secure_record.py::test_malformed_record[short] PASSED [ 74%]
tests/test_secure_record.py::test_malformed_record[truncated] PASSED [ 77%]
tests/test_secure_record.py::test_malformed_record[extra] PASSED [ 80%]
tests/test_secure_record.py::test_authenticated_invalid_header[0-2-unsupported record version] PASSED [ 82%]
tests/test_secure_record.py::test_authenticated_invalid_header[1-1-wrong record direction] PASSED [ 85%]
tests/test_secure_record.py::test_authenticated_wrong_iv PASSED [ 88%]
tests/test_secure_record.py::test_empty_plaintext PASSED [ 91%]
tests/test_secure_record.py::test_sequence_and_iv_encoding PASSED [ 94%]
tests/test_secure_record.py::test_send_sequence_exhaustion PASSED [ 97%]
tests/test_secure_record.py::test_separate_directional_keys PASSED [100%]

===== 35 passed in 78.17s (0:01:18) =====

```